Objects and Operations That Span Requests

While the bulk of the state that persists across requests belongs in the database and is accessed via Active Record, some other bits of state have different life spans and need to be managed differently. In the Depot application, while the Cart itself was stored in the database, knowledge of which cart is the current cart was managed by sessions. Flash notices were used to communicate messages such as “Can’t delete the last user” to the next request after a redirect. And callbacks were used to extract locale data from the URLs themselves.

In this section, we’ll explore each of these mechanisms in turn.

Rails Sessions

A Rails session is a hash-like structure that persists across requests. Unlike raw cookies, sessions can hold any objects (as long as those objects can be marshaled), which makes them ideal for holding state information in web applications. For example, in our store application, we used a session to hold the shopping cart object between requests. The `Cart` object could be used in our application just like any other object. But Rails arranged things such that the cart was saved at the end of handling each request and, more important, that the correct cart for an incoming request was restored when Rails started to handle that request. Using sessions, we can pretend that our application stays around between requests.

And that leads to an interesting question: exactly where does this data stay around between requests? One choice is for the server to send it down to the client as a cookie. This is the default for Rails. It places limitations on the size and increases the bandwidth but means that there’s less for the server to manage and clean up. Note that the contents are (by default) encrypted, which means that users can neither see nor tamper with the contents.